

Aspect Oriented Programming (AOP)

 by Praveen Kumar

What is AOP?

Aspect Oriented Programming is a programming paradigm that allows you to:

- Define **cross-cutting concerns** separately
- Apply them **declaratively** across the application

In Spring, AOP is implemented using:

- **Proxy-based AOP**
- **Annotations or XML configuration**

Why AOP? (The Core Problem)

In real applications, some logic is **needed everywhere**:

- Logging
- Security checks
- Transaction management
- Performance monitoring
- Exception handling

These are called **cross-cutting concerns**.

Problem without AOP

- Same code repeated in many methods
- Difficult to maintain
- Business logic gets polluted

👉 **AOP solves this by separating cross-cutting concerns from business logic.**

Core AOP Terminology (Very Important)

Term	Meaning
Aspect	A class containing cross-cutting logic
Advice	Code executed at a specific point
Join Point	A method execution point
Pointcut	Expression that selects join points
Weaving	Linking aspects with business code
Target Object	Actual business object
Proxy	Wrapper created by Spring

Types of Advices in Spring AOP

Spring supports **5 types of advices**:

Advice	When it runs
@Before	Before method execution
@After	After method execution (always)
@AfterReturning	After successful return
@AfterThrowing	When exception is thrown
@Around	Before + after + control execution

Simple Business Use Case

Business Requirement

We have a **BankService**:

- Deposit money
- Withdraw money

We want:

- Logging
- Execution time tracking
- Exception logging

Business Class (Target Object)

```
@Service
public class BankService {

    public void deposit(int amount) {
        System.out.println("Depositing amount: " + amount);
    }

    public void withdraw(int amount) {
        System.out.println("Withdrawing amount: " + amount);
        if (amount > 5000) {
            throw new RuntimeException("Insufficient balance");
        }
    }
}
```

👉 No logging, no security, no timing code here

✓ Clean business logic

Creating an Aspect Class

```
@Aspect
@Component
public class BankAspect {
```

This class will contain **all advices**.

8. @Before Advice (Pre-processing)

Use Case

- Logging
- Security checks
- Validation

```
@Before("execution(* com.example.service.BankService.*(..))")
public void beforeAdvice() {
    System.out.println("Before Advice: Method execution started");
}
```

📌 Executes **before every method** in `BankService`

@After Advice (Finally Block)

Runs:

- Always (success or exception)

```
@After("execution(* com.example.service.BankService.*(..))")
public void afterAdvice() {
    System.out.println("After Advice: Method execution completed");
}
```

 Similar to `finally` block

@AfterReturning Advice (Success Case)

Runs:

- Only if method executes successfully

```
@AfterReturning(  
    pointcut = "execution(* com.example.service.BankService.deposit(..))",  
    returning = "result"  
)  
public void afterReturningAdvice(Object result) {  
    System.out.println("After Returning Advice: Method completed successfully");  
}
```

✓ Ideal for:

- Audit logging
- Success notifications

@AfterThrowing Advice (Exception Case)

Runs:

- Only when exception occurs

```
@AfterThrowing(  
    pointcut = "execution(* com.example.service.BankService.withdraw(..))",  
    throwing = "ex"  
)  
public void afterThrowingAdvice(Exception ex) {  
    System.out.println("After Throwing Advice: Exception occurred -> " + ex.getMessage());  
}
```

✓ Ideal for:

- Error logging
- Alerting
- Rollback logic

@Around Advice (Most Powerful)

Why @Around is special?

- Can run **before and after**
- Can **control method execution**
- Can **modify return value**

```
@Around("execution(* com.example.service.BankService.*(..))")
public Object aroundAdvice(ProceedingJoinPoint pjp) throws Throwable {

    System.out.println("Around Advice: Before method");

    long startTime = System.currentTimeMillis();

    Object result = pjp.proceed(); // method execution

    long endTime = System.currentTimeMillis();

    System.out.println("Around Advice: After method");
    System.out.println("Execution time: " + (endTime - startTime) + " ms");

    return result;
}
```

✓ Used for:

- Performance monitoring
- Transactions
- Caching
- Security

Execution Order (Very Important Interview Topic)

When all advices are applied:

1. `@Around` (before part)
2. `@Before`
3. **Target Method**
4. `@AfterReturning` (if success)
5. `@AfterThrowing` (if exception)
6. `@After`
7. `@Around` (after part)

Pointcut Expressions Explained

Common Syntax

```
execution(modifiers return-type package.class.method(args))
```

Examples

Expression	Meaning
<code>execution(* *.*(..))</code>	All methods
<code>execution(* service.*.*(..))</code>	All service methods
<code>execution(* withdraw(..))</code>	Only withdraw
<code>execution(* *(int))</code>	Methods with int parameter

Spring AOP vs AspectJ (Interview)

Spring AOP	AspectJ
Runtime proxy-based	Compile / Load time
Only method level	Field, constructor, method
Simpler	More powerful
Default in Spring	External dependency

How Spring AOP Works Internally

1. Spring creates a **proxy object**
2. Proxy wraps the target object
3. Method calls go through proxy
4. Advices are applied
5. Control passes to target method

Framework used: **Spring Framework**

Real-Time Use Cases of AOP

- ✓ Logging framework
- ✓ Spring Transactions (@Transactional)
- ✓ Spring Security
- ✓ Metrics & Monitoring
- ✓ Auditing
- ✓ Retry mechanisms